処理フロー索引（Flow Index）

このファイルは「どのルートが、どのクラスを、どの順で通るか」を最短で追うための索引。

読み方

1. `Route`（入口）を特定する
2. `Controller` の該当メソッドへ移動する
3. そのメソッドが呼ぶ `Workflow` / `Query` / `Service` / `Presenter` / `ViewModel` を順に追う
4. 最後に `View` または `Redirect` を確認する
5. `主な副作用` で更新対象テーブルを確認する

---

1. 認証・入口

1-1. ルート `/`（name: `root`）

- Route: `GET /`
- 実行順:

1. `routes/web.php` のクロージャ
2. 未ログインなら `login` へリダイレクト
3. 管理者なら `admin.dashboard` へリダイレクト
4. メール未認証なら `verification.notice` へリダイレクト
5. それ以外は `attendance.index` へリダイレクト

- 主な副作用: なし（参照とリダイレクトのみ）

1-2. 管理者ログイン

- Route: `GET /admin/login`（`admin.login`）
- Route: `POST /admin/login`（`admin.login.store`）
- 実行順:

1. `AuthenticatedSessionController@create/store`（Fortify）
2. Fortifyの認証ロジック（`FortifyServiceProvider`）
3. 成功時はログインレスポンスに従って遷移

- 主な副作用: `sessions`（セッションストア設定に依存）

1-3. ログアウト

- Route: `POST /logout`（`logout`）
- 実行順:

1. `AuthenticatedSessionController@destroy`（Fortify）
2. セッション破棄
3. リダイレクト

- 主な副作用: `sessions`（セッション削除）

---

2. 一般ユーザー勤怠（打刻）

2-1. 打刻トップ表示

- Route: `GET /attendance`（`attendance.index`）
- 実行順:

1. `AttendanceController@index`
2. 当日 `Attendance` を取得
3. `view('attendance', [...])`

- 主な副作用: なし（参照のみ）

2-2. 出勤打刻

- Route: `POST /attendance/in`（`attendance.in`）
- 実行順:

1. `AttendanceController@checkIn`
2. `AttendanceWorkflow::stamp(userId, 'check_in')`
3. `redirect()->route('attendance.index')`

- 主な副作用: `attendances`（当日レコード作成/更新）

2-3. 退勤打刻

- Route: `POST /attendance/out`（`attendance.out`）
- 実行順:

1. `AttendanceController@checkOut`
2. `AttendanceWorkflow::stamp(userId, 'check_out')`
3. `redirect()->route('attendance.index')`

- 主な副作用: `attendance_breaks`（未終了休憩の終了時刻更新）, `attendances`（退勤時刻・状態更新）

2-4. 休憩入

- Route: `POST /attendance/break_in`（`attendance.break_in`）
- 実行順:

1. `AttendanceController@breakIn`
2. `AttendanceWorkflow::stamp(userId, 'break_in')`
3. `redirect()->route('attendance.index')`

- 主な副作用: `attendance_breaks`（休憩開始作成）, `attendances`（状態更新）

2-5. 休憩戻

- Route: `POST /attendance/break_out`（`attendance.break_out`）
- 実行順:

1. `AttendanceController@breakOut`
2. `AttendanceWorkflow::stamp(userId, 'break_out')`
3. `redirect()->route('attendance.index')`

- 主な副作用: `attendance_breaks`（最新未終了休憩の終了時刻更新）, `attendances`（状態更新）

---

3. 一般ユーザー勤怠（一覧・詳細・修正申請）

3-1. 勤怠一覧（月次）

- Route: `GET /attendance/list`（`attendance.list`）
- 実行順:

1. `AttendanceController@list`
2. `AttendanceListQuery::forUserMonth(... includeMissingDates=true)`
3. `DateNavigationPresenter::forMonth(...)`
4. `AttendanceListScreenPresenter::forUser(...)`
5. `view('attendance_records_screen', ...)`

- 主な副作用: なし（参照のみ）

3-2. 勤怠詳細（ID指定）

- Route: `GET /attendance/detail/{attendance}`（`attendance.detail`）
- 実行順:

1. `AttendanceController@detail`
2. Policy `AttendancePolicy@view`
3. `attendance + breaks` をロード
4. `view('attendance_detail_screen', ...)`
5. 画面内部で `AttendanceDetailViewModelFactory` により入力表示データを構築

- 主な副作用: なし（参照のみ）

3-3. 勤怠詳細（日付指定）

- Route: `GET /attendance/detail/date/{date}`（`attendance.detail.date`）
- 実行順:

1. `AttendanceController@detailByDate`
2. 日付文字列を検証
3. 当日 `Attendance` を `firstOrCreate`
4. `AttendanceController@detail` へ委譲

- 主な副作用: `attendances`（対象日のレコード新規作成の可能性）

3-4. 修正申請の作成

- Route: `PUT /attendance/request/{attendance}`（`attendance.request`）
- 実行順:

1. `CorrectionController@store`
2. `AttendanceCorrectionRequest` で入力検証
    - 追加検証: 「休憩終了のみ入力」はエラー（`break_end_at.{index}`）
3. Policy `AttendancePolicy@store`
4. `AttendanceWorkflow::requestCorrection(attendance, requestUserId, payload)`
5. `AttendanceDetailViewModelFactory::buildFromCorrection(...)`
6. `response()->view('attendance_detail_screen', 承認待ち表示)`

- 主な副作用: `attendance_corrections`（申請作成）, `break_corrections`（休憩修正行作成）

3-5. 修正申請の詳細表示

- Route: `GET /stamp_correction_request/{attendanceCorrection}`（`stamp_correction_request.detail`）
- 実行順:

1. `CorrectionController@detail`
2. Policy `AttendanceCorrectionPolicy@view`
3. `AttendanceDetailViewModelFactory::buildFromCorrection(...)`
4. 承認状態で編集可否を分岐
5. `view('attendance_detail_screen', ...)`

- 主な副作用: なし（参照のみ）

3-6. 申請一覧（一般・管理者共通入口）

- Route: `GET /stamp_correction_request/list`（`stamp_correction_requests.list`）
- 実行順:

1. `StampCorrectionListController@index`
2. 管理者なら `AdminCorrectionController@list` へ
3. 一般ユーザーで未認証メールなら `verification.notice` へ
4. それ以外は `CorrectionController@list` へ

- 主な副作用: なし（参照と分岐のみ）

---

4. 管理者勤怠

4-1. 管理者ダッシュボード（日次勤怠一覧）

- Route: `GET /admin/attendance/list`（`admin.dashboard`）
- 実行順:

1. `AdminAttendController@index`
2. `AttendanceListQuery::forDay(...)`
3. `DateNavigationPresenter::forDay(...)`
4. `AttendanceListScreenPresenter::forAdmin(...)`

5. `view('attendance_records_screen', ...)`

- 主な副作用: なし（参照のみ）

4-2. 管理者の勤怠詳細

- Route: `GET /admin/attendance/{attendance}`（`admin.attendance.detail`）
- 実行順:

1. `AdminAttendController@detail`
2. Policy `AttendancePolicy@view`
3. `attendance + breaks` をロード
4. `view('attendance_detail_screen', ...)`

- 主な副作用: なし（参照のみ）

4-3. 管理者の勤怠更新

- Route: `PUT /admin/attendance/{attendance}`（`admin.attendance.update`）
- 実行順:

1. `AdminAttendController@update`
2. `AttendanceCorrectionRequest` で入力検証
    - 追加検証: 「休憩終了のみ入力」はエラー（`break_end_at.{index}`）
3. Policy `AttendancePolicy@update`
4. `AdminAttendUpdateService::update(...)`
5. `redirect()->route('admin.attendance.detail')`

- 主な副作用: `attendances`（出退勤・備考更新）, `attendance_breaks`（全削除後再作成）

4-4. スタッフ一覧

- Route: `GET /admin/staff/list`（`admin.staff_list`）
- 実行順:

1. `AdminAttendController@staff`
2. 一般ユーザーのみ取得
3. `view('admin_attendance_staff', ...)`

- 主な副作用: なし（参照のみ）

4-5. スタッフ月次勤怠一覧

- Route: `GET /admin/attendance/staff/{user}`（`admin.attendance.list`）
- 実行順:

1. `AdminAttendController@staffAttendances`
2. `AttendanceListQuery::forUserMonth(... withUser=true)`
3. `DateNavigationPresenter::forMonth(...)`
4. `AttendanceListScreenPresenter::forAdmin(..., csvUrl)`
5. `view('attendance_records_screen', ...)`

- 主な副作用: なし（参照のみ）

4-6. スタッフ月次勤怠CSV

- Route: `GET /admin/attendance/staff/{user}/csv`（`admin.attendance.list.csv`）
- 実行順:

1. `AdminAttendController@staffCsv`
2. `AttendanceListQuery::forUserMonth(...)`
3. `response()->streamDownload(...)`
4. CSVを書き出して返却

- 主な副作用: なし（参照＋レスポンス出力のみ）

---

5. 管理者申請承認

5-1. 申請一覧（管理者）

- Route: `GET /stamp_correction_request/list`（管理者分岐時）
- 実行順:

1. `AdminCorrectionController@list`
2. `AttendanceCorrection` 一覧取得（requestUser 付き）
3. `AttendanceListScreenPresenter::forApplicationAdmin(...)`
4. `view('applications_screen', ...)`

- 主な副作用: なし（参照のみ）

5-2. 申請承認画面

- Route: `GET /stamp_correction_request/approve/{attendanceCorrection}`（`admin.attendance.approve`）
- 実行順:

1. `AdminCorrectionController@detail`
2. Policy `AttendanceCorrectionPolicy@view`
3. `AttendanceDetailViewModelFactory::buildFromCorrection(...)`
4. `view('attendance_detail_screen', 承認ボタン表示)`

- 主な副作用: なし（参照のみ）

5-3. 申請承認実行

- Route: `PUT /stamp_correction_request/approve/{attendanceCorrection}`（`admin.attendance.approve.update`）
- 実行順:

1. `AdminCorrectionController@approve`
2. Policy `AttendanceCorrectionPolicy@approve`
3. `AttendanceWorkflow::approveCorrection(attendanceCorrection, adminUserId)`

4. `redirect()->route('stamp_correction_requests.list')`

- 主な副作用: `attendances`（申請内容反映）, `attendance_breaks`（必要時に全削除後再作成）, `attendance_corrections`（承認情報更新）

---

6. 主要業務クラスの役割

- `AttendanceWorkflow`
  - 打刻 (`stamp`)
  - 修正申請作成 (`requestCorrection`)
  - 承認反映 (`approveCorrection`)
- `AttendanceListQuery`
  - 日次/ユーザー月次勤怠を取得
  - 表示用の計算済み時間を付与
- `AdminAttendUpdateService`
  - 管理者による勤怠更新（本体 + 休憩再構築）
- `AttendanceDetailViewModelFactory`
  - 修正申請詳細画面の表示データ構築
  - 勤怠詳細フォームの入力表示行を整形
- `DateNavigationPresenter`
  - 前後日/前後月リンクとラベルを生成
- `AttendanceListScreenPresenter`
  - 勤怠一覧画面の表示配列を生成
  - 申請一覧画面の表示配列を生成
- `StampCorrectionListController`
  - 申請一覧の管理者/一般ユーザー分岐を担当

---

7. 追跡時の最短コマンド

```
# ルート定義確認
nl -ba routes/web.php

# 対象ルート名がどこで使われるか
rg -n "attendance\.list|admin\.dashboard|stamp_correction_requests\.list"
app routes resources

# コントローラから次の呼び出し先を追う
```

```
rg -n "AttendanceWorkflow|AttendanceListQuery|Presenter|ViewModel|Service"
app/Http/Controllers
```